

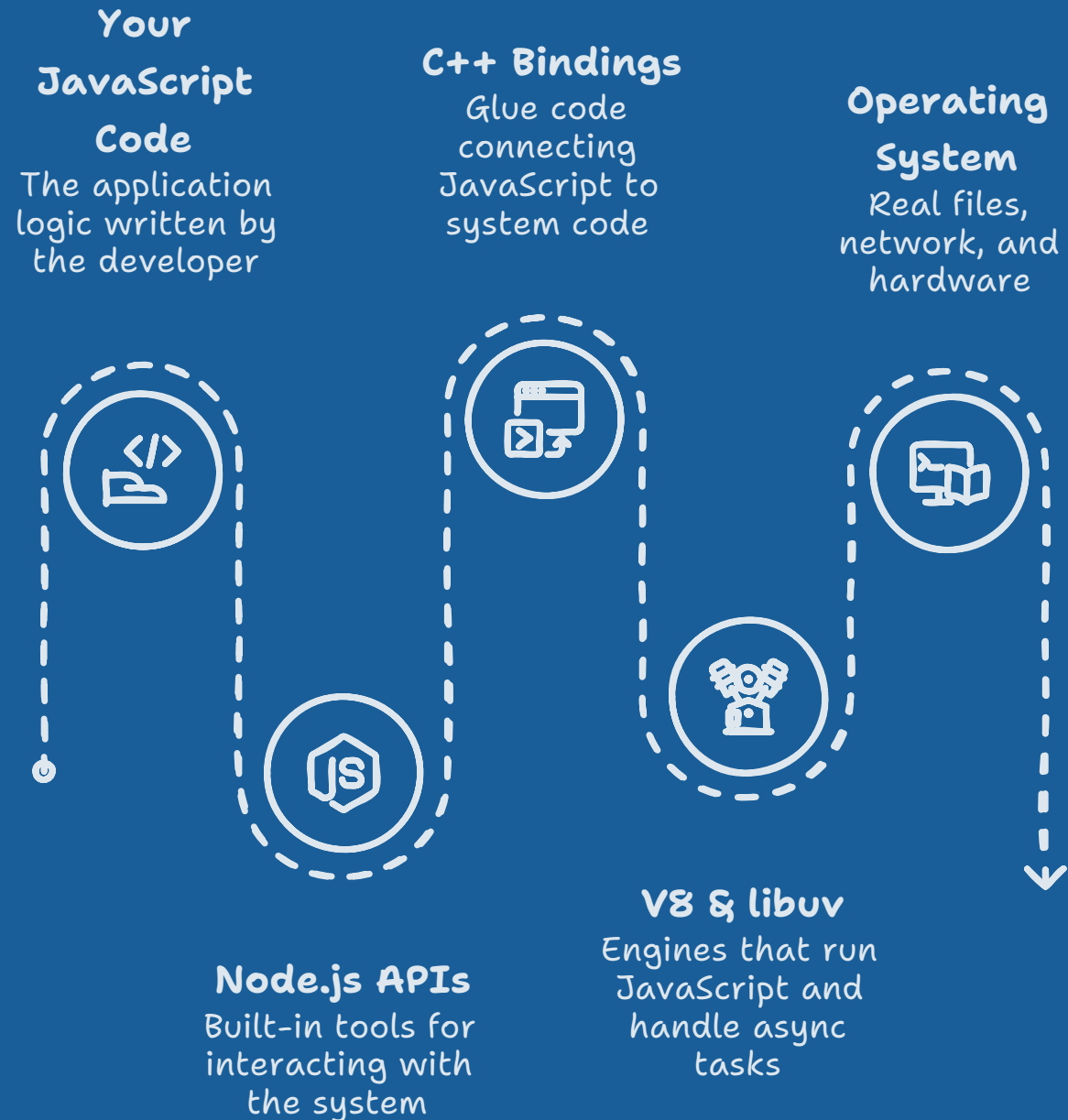
The Node.js Mental Model

A complete guide to how Node.js works under the hood

JavaScript Outside the Browser

The Big Picture: Node.js is a runtime that connects your code to the system

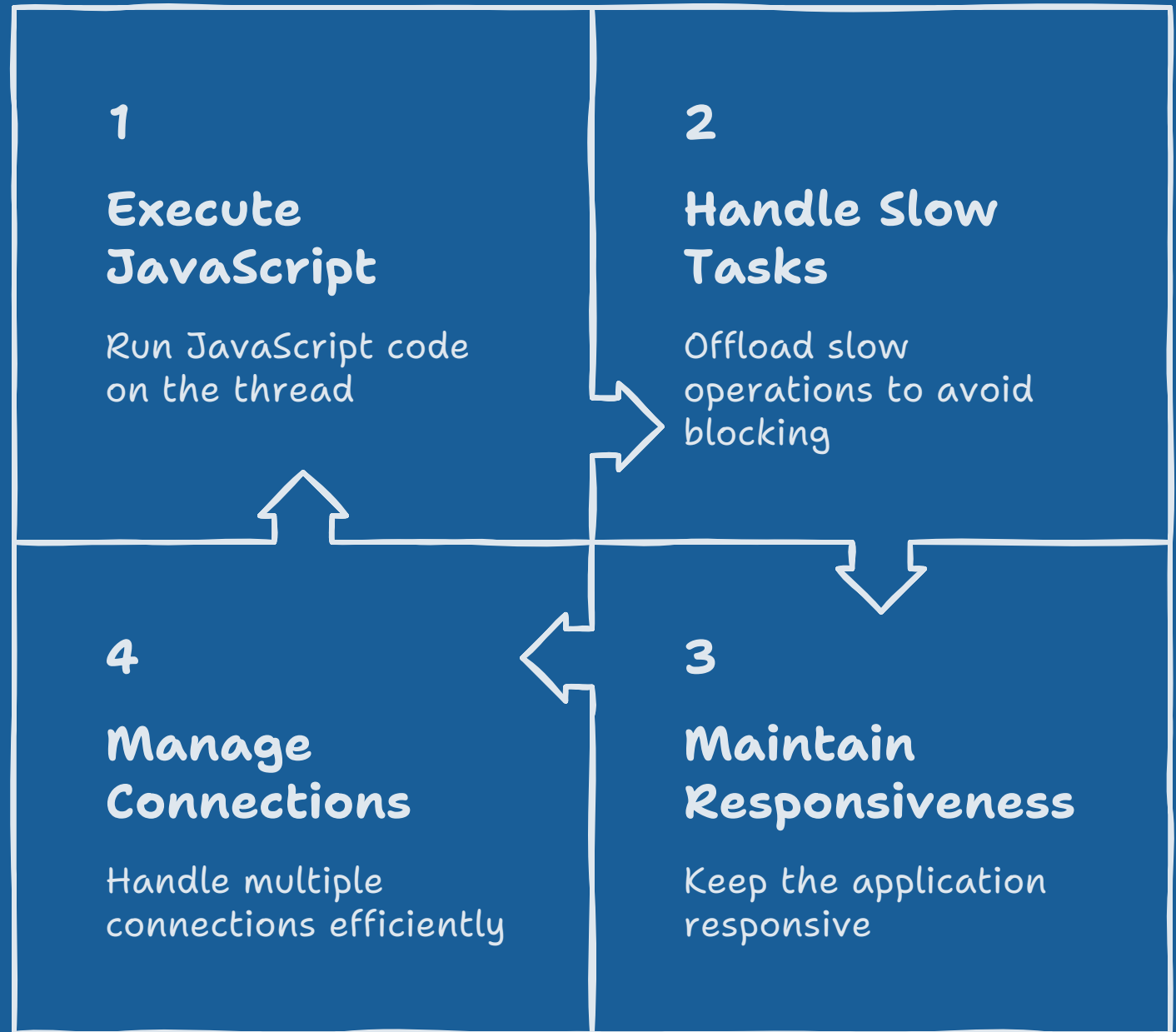
The Node.js stack layers represent the path from your application code down to the operating system.



The Single Thread

Node runs your JS on exactly one thread to keep things simple

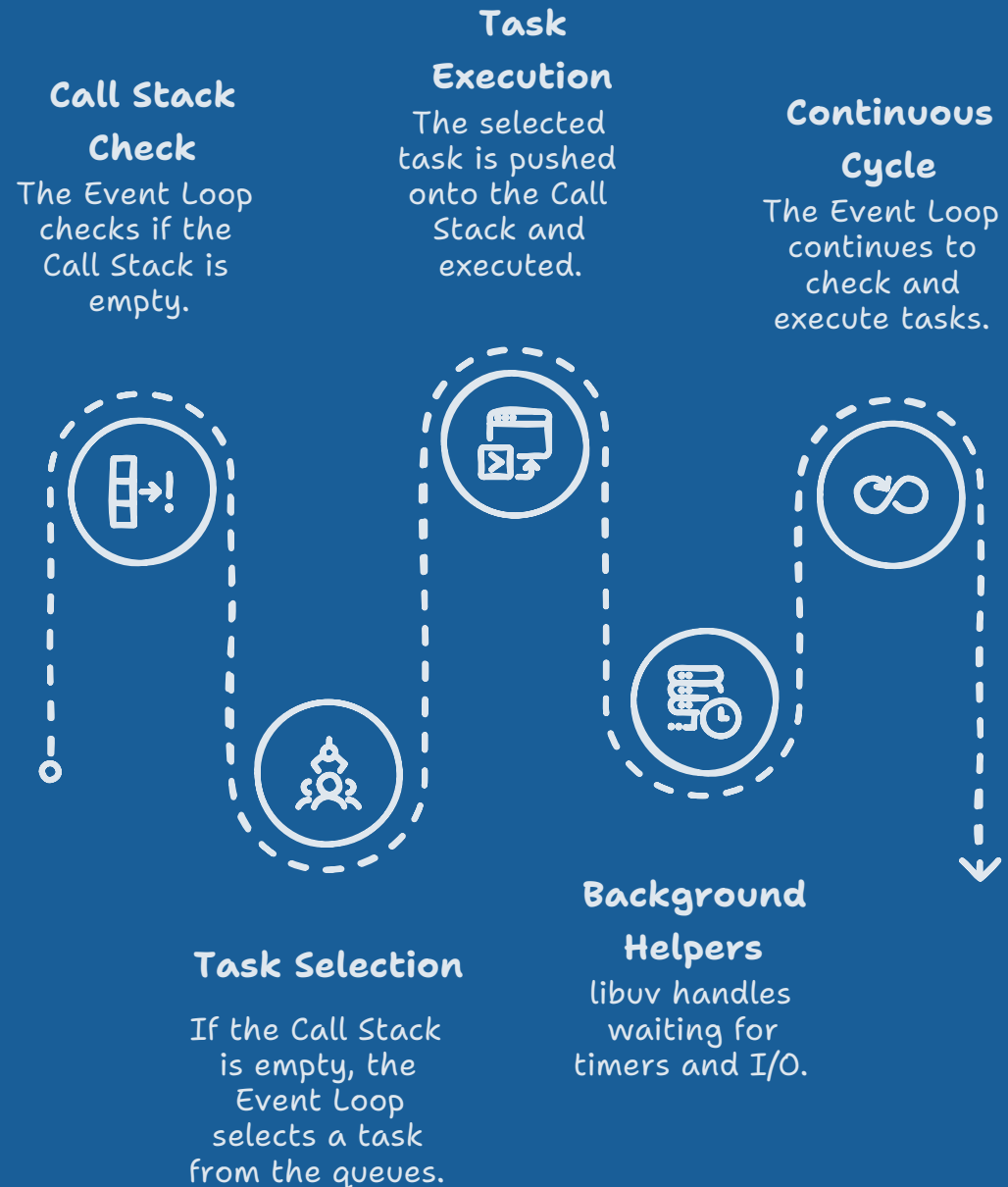
Only one line of your JavaScript executes at any instant. This prevents complex race conditions but requires a special way to handle slow tasks without freezing the entire application.



The Event Loop

The traffic controller deciding what runs next

The Event Loop is a continuous cycle that manages the execution of your code. It constantly checks if the Call Stack is empty and decides which task from the queues should be pushed onto it next. Call Stack: Where your code actually runs, one function at a time. Background Helpers: libuv handles the waiting for timers and I/O. Continuous Check: It never stops for the life of your program.



The Queues

Promises jump the line while timers wait their turn

Callback Queue

setTimeout and I/O (One task per loop)



Microtask Queue

Promises and async/await (Drained completely first)

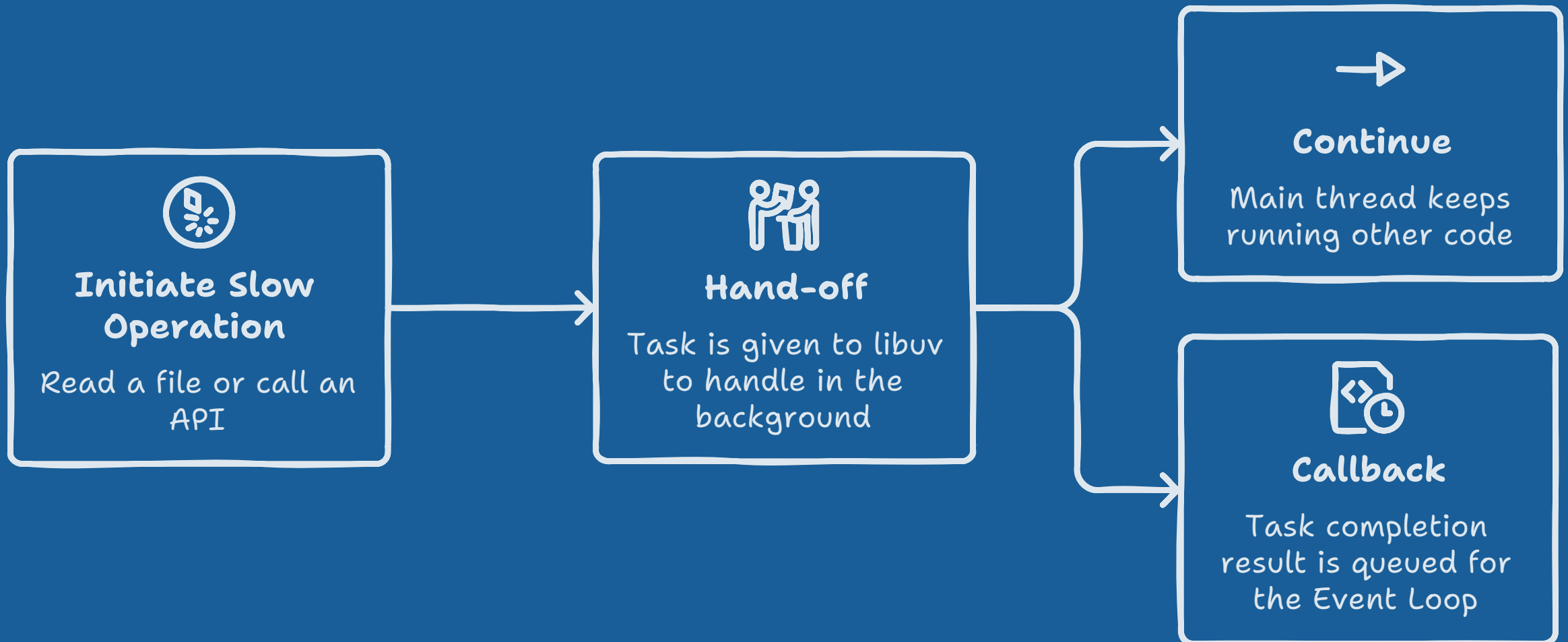


Sync Code

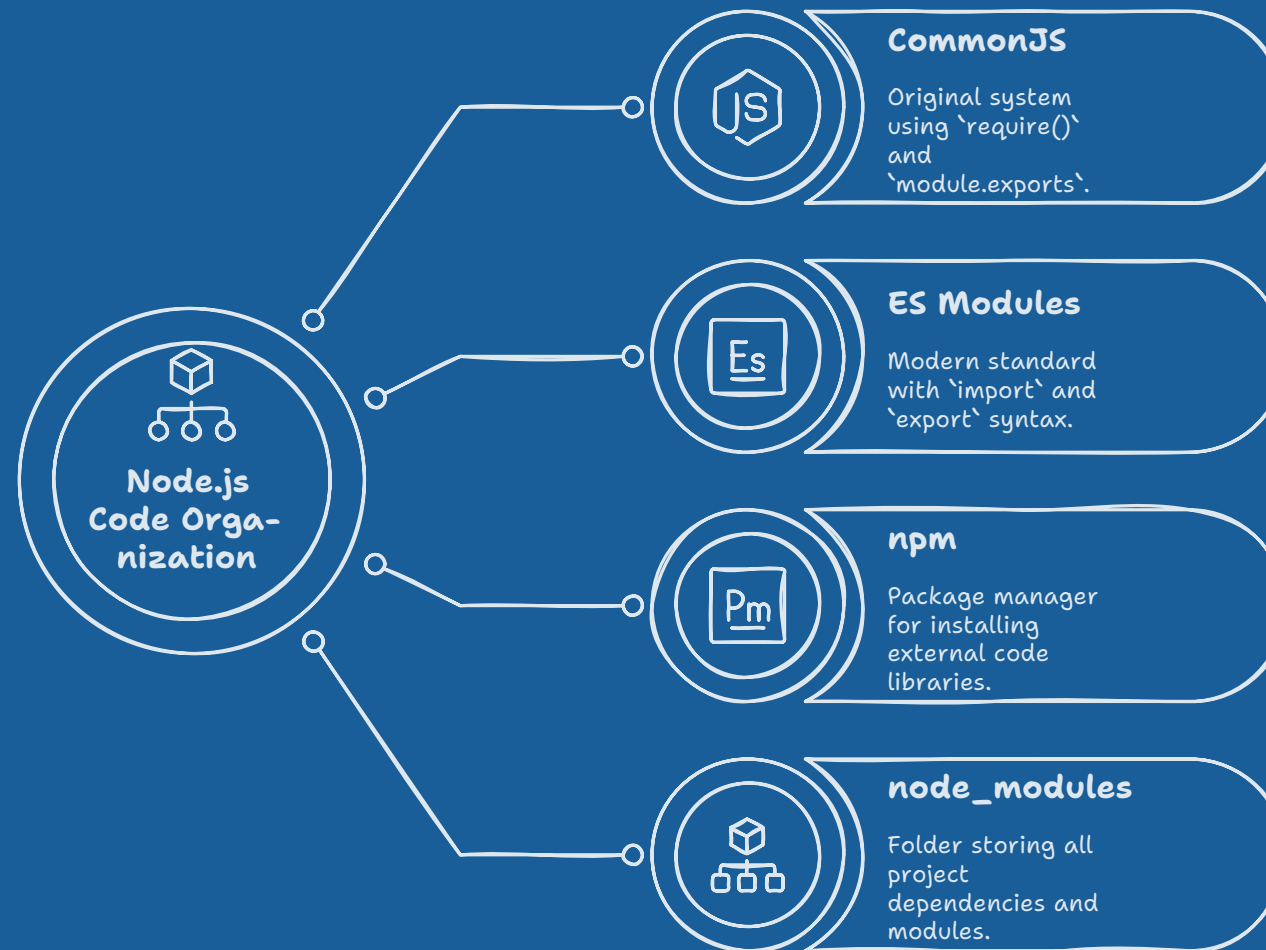
Runs immediately on the Call Stack



Non-Blocking I/O Process in Node.js



Unveiling Node.js Code Organization



Global Tools

Environment: Free tools available in every file without importing



Use ``process`` to get
system information



Use ``__dirname`` to
find the current
directory



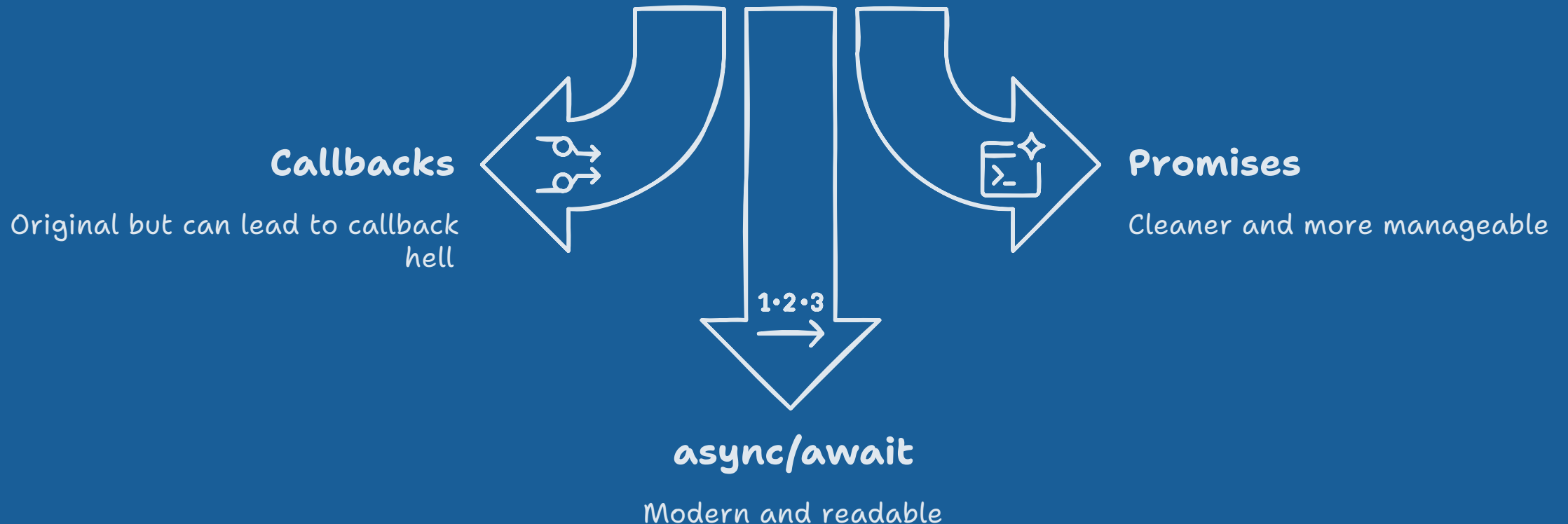
Use ``__filename`` to
find the current file



Use ``setTimeout``
and ``setInterval``
for timing

Async Patterns

Three ways to write the same underlying logic



Anatomy of a Node.js Project

